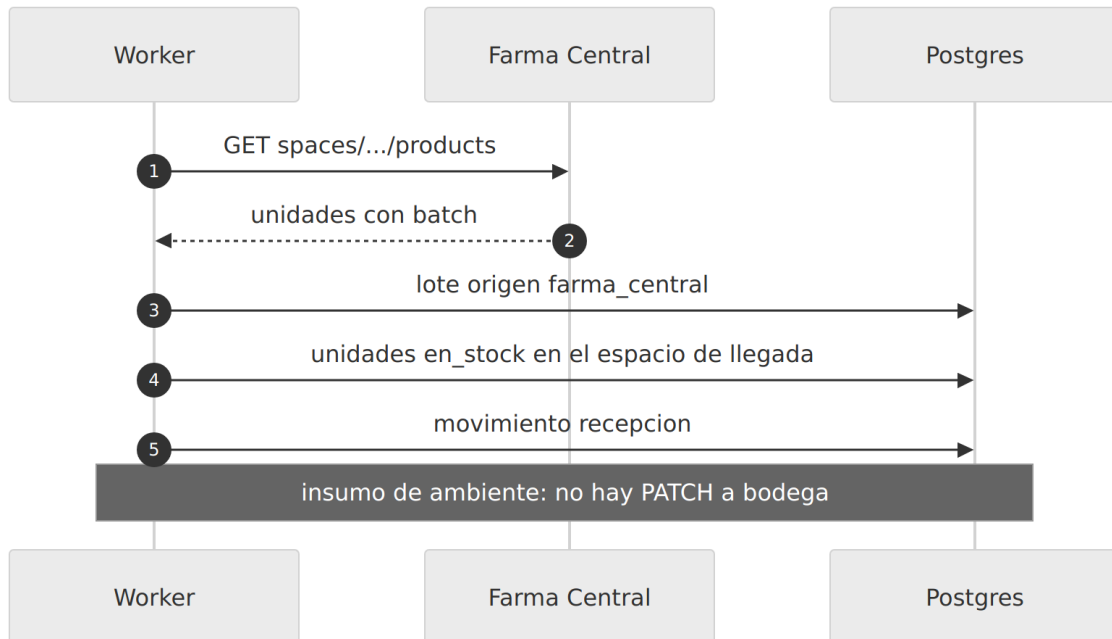
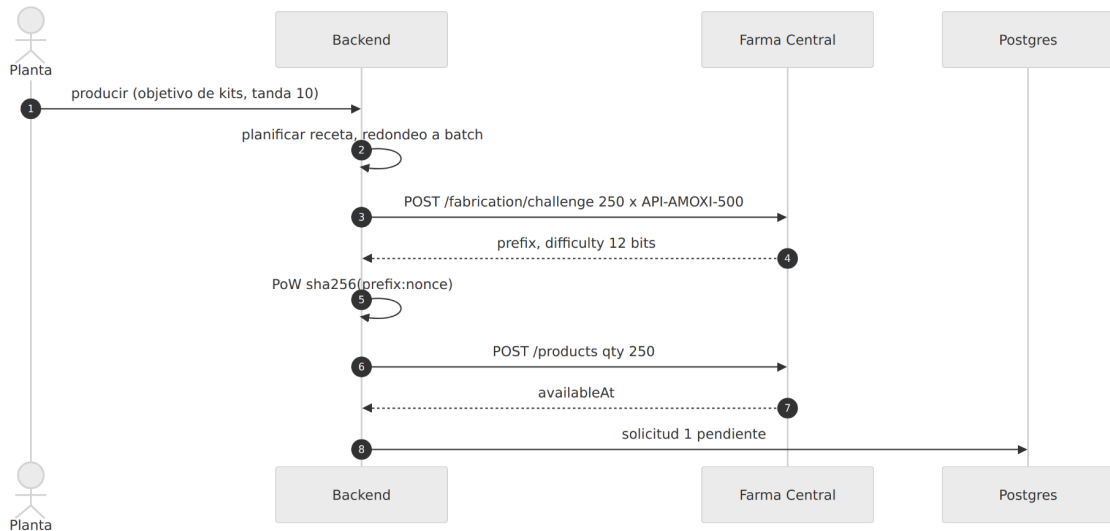


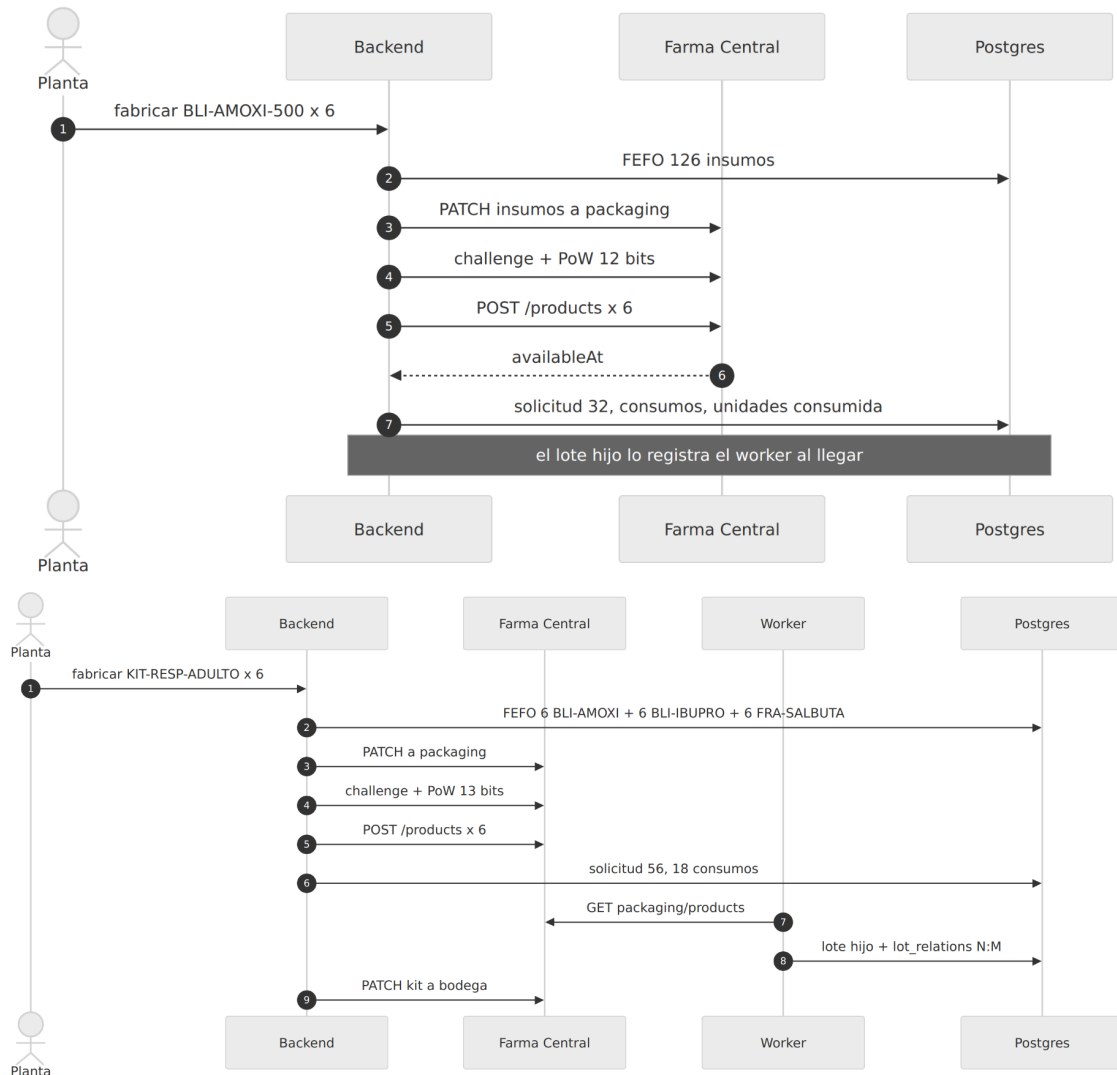
Diseño de la Solución
Arturo Pacheco - Luis Reyes

DIAGRAMAS DE SECUENCIA POR PROCESO:

Abastecimiento API-AMOXI-500: Nuestro código llama a **producir** que es una función que arma la cantidad de kits que indiques en la llamada. Hace la primera compra de lo que calcula de lo necesitado $250 \times \text{API-AMOXI-500}$. Se hace PoW. Farma luego responde availableAt. El lote nace en checkIn y el worker lo mueve a bodega .



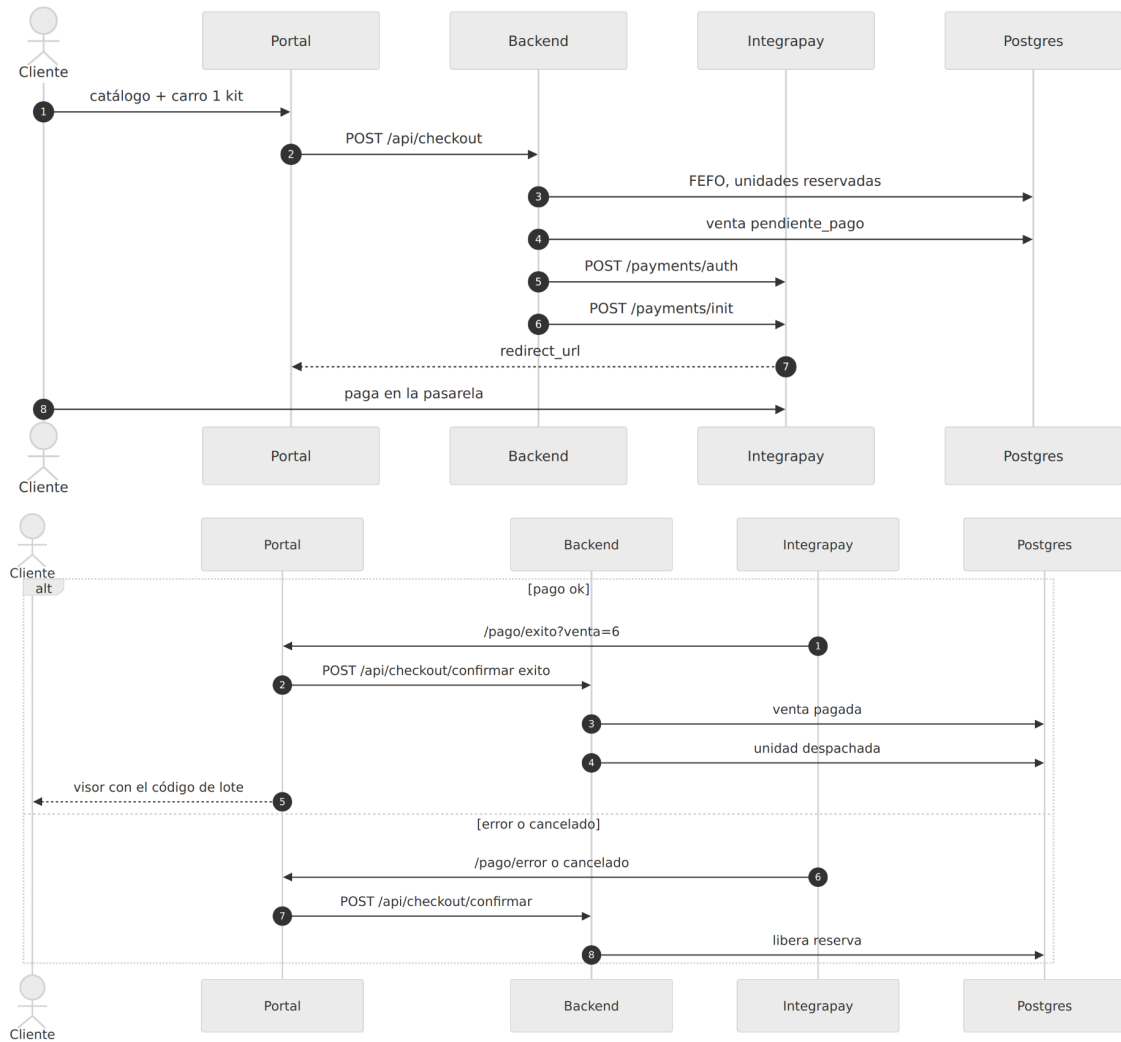
Acondicionamiento y Armado de Kit: Nuestro código llama a **fabricar** que es una función que arma los otros insumos necesarios para armar los kits, luego se fabrican los kits en sí cuando se tienen los insumos necesarios, considerar que el worker mueve los insumos si es necesario y también registra sus llegadas y así.



Consideramos planta como quien da la orden de fabricar, en este caso somos nosotros, luego debería ser automático.

Venta de Kits:

La venta de kits sigue el flujo esperado condicionado por la api.



JUSTIFICACIÓN DE DECISIONES DE DISEÑO:

Patrón de integración con Farma Central

Para integrarnos con Farma Central decidimos centralizar todas las llamadas en un único cliente `farma_client.py`, en lugar de llamar a la API desde distintas partes del sistema. Lo hicimos porque toda llamada a Farma requiere lo mismo, obtener y renovar el token, reintentar si el token venció y esperar si se supera el límite de 250 requests por minuto. Si cada parte resolviera esto por su cuenta, la lógica se repetiría en muchos lugares. Con el mismo criterio, la pasarela de pago quedó detrás de su propio conector

Ganancias

- El token, los reintentos y el manejo del rate limit se escriben una sola vez, y todos los usan igual.
- Si Farma cambia algo (una URL, el formato del token, un header), se corrige en un solo archivo.
- Cambiar entre dev y prod es sólo cambiar variables de entorno.

Pérdidas

- Un error en el conector afecta a todo el sistema a la vez.
- El conector aplica el mismo comportamiento a todos, aunque no a todos les convenga. Por ejemplo, ante un 429 espera y reintentar, a un script de producción le da lo mismo, pero el worker de frío queda bloqueado mientras tanto, cuando le convendría más abandonar y reintentar en el siguiente ciclo.

Estado de los productos en una base de datos propia

Para llevar el estado de los productos decidimos tener en nuestra propia base de datos el estado actual de cada unidad, un historial de eventos que no se modifica y la genealogía entre lotes, en lugar de consultar a Farma cada vez. Lo hicimos porque Farma sabe dónde está cada unidad ahora, pero no guarda nuestra historia interna, y las cuatro preguntas de trazabilidad requieren ese flujo. Además, el visor tiene que responder rápido, sin recorrer todo el historial.

Ganancias

- Podemos responder las cuatro preguntas de trazabilidad, incluidas las de aguas arriba, aguas abajo y clientes.
- El stock del portal y el visor son consultas rápidas a nuestra base y no dependen de que Farma esté disponible.

Pérdidas

- La información queda duplicada respecto de Farma, así que hay que sincronizar, y por momentos ambas pueden no coincidir.
- Cada evento implica dos escrituras: una en el estado y otra en el historial.

Portal síncrono y operaciones asíncronas

Para repartir el trabajo del servidor decidimos que el portal de venta funcione de forma síncrona y que la compra de insumos, la fabricación, el PoW y la cadena de frío corran en un proceso aparte, limitado a un núcleo y con baja prioridad. Lo hicimos porque el servidor tiene solo 2 núcleos compartidos y el PoW puede consumir toda la CPU y botar el portal, mientras que el frío exige reaccionar en minutos, sin esperar a que alguien haga un request. En la venta, como el resultado del pago llega después y la pasarela falla el 20% de las veces, la venta pasa por estados (pendiente_pago, pagada, cancelada) en vez de completarse de una sola vez.

Ganancias

- El portal sigue respondiendo mientras se resuelve el PoW.
- Una caída a mitad del pago no deja la venta en un estado inconsistente: queda pendiente y se puede recuperar.
- La cadena de frío funciona aunque nadie esté usando el portal, porque el worker corre de forma independiente

Pérdidas

- Hay más estados que manejar y hay que recuperar lo que quedó a medias .
- Sin una cola de mensajes, no hay reintentos automáticos entre procesos. Para la E2 podría hacer falta.
- En la E1, la producción todavía la lanza una persona a mano.

ESTRATEGIA DE ABASTECIMIENTO Y PRODUCCIÓN:

Nuestra estrategia es producir por demanda, de a poco. No mantenemos un stock mínimo de cada cosa. El punto de partida es un objetivo de kits, por ejemplo 30 de cada uno, y desde ahí el sistema calcula hacia abajo qué falta.

Cada 20 segundos, producir (funcion en el código en produccion.py) da una vuelta y hace lo que pueda con el estado actual, en este orden:

1. Saca de acondicionamiento los kits que ya nacieron y los lleva a la bodega.
2. Arma los kits para los que ya tiene los intermedios.
3. Fábrica los intermedios para los que ya llegaron los insumos.
4. Compra los insumos que faltan, descontando lo que ya viene en camino.

Si algo se atrasa, se hace en la vuelta siguiente. El cálculo de qué falta lo hace planificar. Parte de los kits, los descompone en intermedios y luego en insumos, a cada nivel le resta lo que ya hay en stock y lo que viene en camino, y redondea al tamaño de lote. Como recorre todos los kits juntos, un insumo compartido se pide una sola vez sumando la demanda de todos.

- **Umbrales de Reposición:** No hay umbrales fijos tipo "si quedan menos de X, compro". Un insumo o intermedio se repone cuando lo que hay, más lo que viene en camino, no alcanza para la tanda actual de kits. El umbral, entonces, es la necesidad de lo que se pide.
- **Cantidades de compra:** Se compra exactamente lo que falta para lo que se pidió, redondeado hacia arriba al tamaño de lote. Lo que sobra por el redondeo queda para la tanda siguiente, y planificar lo descuenta. La tanda es de 10 kits por defecto, no de 30 de una vez. Así no se piden compras gigantes que terminen en la bodega externa que cobra.
- **Gestión de la bodega externa:** Se usa como algo que se usa solo cuando no queda otra. Tenemos tres casos, cuando Farma envía ahí lo que no cabe en recepción o en acondicionamiento cuando la cámara de frío está llena y llega un producto frío, y mientras la bodega principal no tenga espacio para recibir lo que está en la externa. En cada vuelta, el sistema saca a la bodega principal todo lo que no requiere frío si hay espacio, función liberar_buffer en el código, así que el costo se paga solo mientras no exista otra opción.
- **Vencimiento:** Al fabricar, se eligen unidades por FEFO, es decir las primeras en expirar son las que se usan. Iniciamos con las que ya están en acondicionamiento porque no hay que moverlas, y después las que vencen antes. Las frías se mueven a packaging solo al final, para pasar menos tiempo fuera de frío. Si el PoW o el movimiento falla, esas frías se devuelven a cámara o a buffer.
- **Sensibilidad al precio:** Actualmente, como en esta entrega solo se necesitaba crear los 30 kits de cada uno, el precio no afectó la producción. Solo usamos el precio para mostrarlo en el portal de venta. Para el futuro, si el precio de venta de un kit sube un 50%, adelantamos el siguiente pedido si no necesitamos usar la bodega externa. Si baja un 50% y el margen sigue siendo positivo producimos solo lo pedido. Pero si el margen queda negativo, dejamos de fabricar ese kit y vendemos el stock existente. Si el costo de un insumo sube un 50%, compramos solo lo que necesita la tanda actual y recalculamos el margen. Si baja un 50%, compramos hasta dos tandas por adelantado, siempre que el insumo no requiera frío, quepa en la bodega principal y no venza antes de usarse.

MODELO DE CUSTODIA Y TRAZABILIDAD:

Para la trazabilidad decidimos mantener nuestra propia base de datos en vez de depender solamente de Farma. Farma nos permite saber dónde está una unidad actualmente, pero no guarda toda nuestra historia interna, por ejemplo de qué lotes salió un producto acondicionado o qué unidades concretas terminaron en una venta. Como esa información después se necesita para reconstruir la trazabilidad, la registramos nosotros.

Para evitar que cada parte del sistema escriba por su cuenta, todo lo relacionado con custodia queda centralizado en custodia.py. Las operaciones de compra, fabricación y movimiento pasan por operaciones.py, de forma que cada cambio importante también deje su registro correspondiente.

La trazabilidad sigue el mismo flujo que sigue el producto en la distribuidora: primero llega un insumo, después ese insumo puede utilizarse para fabricar un producto acondicionado, luego ese producto puede formar parte de un kit y finalmente las unidades del kit pueden terminar asociadas a una venta.

Qué se registra y cuándo

No guardamos una copia completa de todo lo que tiene Farma. Nuestra base guarda los cambios que son necesarios para reconstruir el recorrido de los productos.

Cuando el worker detecta unidades nuevas provenientes de una compra, registra el lote con origen Farma y deja las unidades en el espacio donde fueron recibidas. Si el lote apareció producto de una fabricación nuestra, se registra como un lote generado y no como una recepción normal.

Los movimientos entre espacios se registran después de que Farma acepta el traslado. Por ejemplo, cuando una unidad pasa desde checkin a bodega o desde recepción hacia la cámara de frío.

En una fabricación también registramos qué unidades fueron utilizadas. Cuando Farma acepta producir, las unidades seleccionadas para esa fabricación quedan marcadas como consumidas y después, cuando aparece el nuevo lote, se registra la relación entre los lotes utilizados y el lote generado.

En las ventas primero se reservan unidades concretas antes de iniciar el pago con Integrapay. Si el pago termina correctamente, esas mismas unidades quedan asociadas al pedido y al comprador. Si el pago falla o se cancela, la reserva se libera.

El modelo también contempla vencimiento y cuarentena, aunque en esta entrega todavía no tenemos automatizado ese flujo en el worker.

De la solicitud al lote

Cuando compramos o fabricamos, Farma no entrega las unidades inmediatamente. Primero responde con una hora availableAt, indicando cuándo deberían aparecer.

Por eso guardamos una solicitud pendiente con la información necesaria, como el SKU, la cantidad solicitada, la hora de llegada y, cuando corresponde, los datos asociados al desafío de fabricación.

Cada 20 segundos el worker revisa los espacios de Farma. Cuando encuentra unidades nuevas con un código de lote, intenta relacionarlas con una solicitud pendiente del mismo SKU cuya hora de llegada ya haya pasado.

La idea es no inventar relaciones. Si aparece un lote que nuestro sistema no puede asociar a una solicitud conocida, se deja como un caso a revisar en vez de asumir automáticamente de dónde salió.

Creación de productos acondicionados y kits

Una fabricación deja registro en dos momentos.

Primero, antes de fabricar, el sistema selecciona las unidades que se van a utilizar usando FEFO y las mueve a acondicionamiento cuando sea necesario. Cuando Farma acepta la fabricación, esas unidades quedan registradas como consumidas.

En ese momento todavía no tenemos el producto nuevo en nuestra base, porque Farma solamente nos entrega la hora en que estará disponible.

Después, cuando el worker detecta las unidades nuevas, crea el lote hijo y registra qué lotes fueron utilizados para generarlo.

Esta relación no puede representarse con un único lote padre. Un blíster puede utilizar unidades provenientes de varios lotes y, al mismo tiempo, un mismo lote de API puede utilizarse en más de una fabricación. Por eso la relación entre lotes consumidos y lotes generados es de muchos a muchos.

Modelo de datos

Para poder reconstruir la historia no basta con guardar solamente el lote.

Guardamos:

- las solicitudes realizadas a Farma
- los lotes
- las unidades individuales
- el estado y ubicación actual de cada unidad
- qué unidades se consumieron durante una fabricación
- la relación entre lotes padre e hijo

- una bitácora de eventos y movimientos
- los pedidos y pagos
- las unidades concretas asociadas a cada venta

El estado actual de las unidades se utiliza para cosas operativas, como mostrar stock en el portal o decidir qué se puede fabricar.

El historial y las relaciones entre lotes se utilizan para reconstruir la trazabilidad.

Cómo respondemos las cuatro preguntas de trazabilidad

Las consultas se concentran en trazabilidad.py y parten desde un código de lote. La idea es utilizar las relaciones e índices de la base y no recorrer todo el historial para cada búsqueda.

1. ¿Qué unidades del lote X tenemos actualmente y dónde están?

Buscamos las unidades asociadas al lote que todavía estén disponibles o reservadas y mostramos el espacio actual de cada una.

2. ¿En qué productos se consumieron unidades del lote X y qué lotes se generaron?

Recorremos las relaciones entre lote padre y lote hijo hacia abajo, siguiendo los productos que fueron generados a partir de ese lote.

3. ¿A qué clientes se entregaron unidades del lote X, cuándo y en qué pedido?

Buscamos las unidades de ese lote que quedaron asociadas a ventas completadas y desde ahí obtenemos el pedido y el comprador. Una reserva o un pago fallido no se considera una entrega.

4. ¿De qué lotes se produjo el lote X y de dónde venían?

Recorremos las mismas relaciones en el sentido contrario, desde el lote hijo hacia sus lotes padre, hasta llegar a los lotes que originalmente fueron recibidos desde Farma u otra distribuidora.

Qué pasa si algo se cae a la mitad

Uno de los problemas que queremos evitar es que Farma tenga una operación realizada pero nuestra base quede con la mitad de la información, o que una venta quede asociada a un producto sin tener registrado a quién se entregó.

En la venta, las unidades primero quedan reservadas. Si Integrapay no logra iniciar el pago o devuelve error, la venta se marca con error y las unidades se liberan.

Si el cliente entra al portal de pago pero no vuelve, las unidades quedan reservadas. Actualmente todavía no tenemos un tiempo máximo que libere automáticamente esas reservas, por lo que es un caso pendiente de mejorar.

Si el pago ya fue confirmado y el proceso se cae antes de terminar el flujo, la confirmación está pensada para ser idempotente, por lo que puede procesarse nuevamente sin duplicar la venta. El worker puede continuar posteriormente con el despacho pendiente.

Para la fabricación usamos un candado al momento de seleccionar unidades, de forma que dos procesos no puedan utilizar al mismo tiempo las mismas unidades.

En esta entrega tampoco movemos automáticamente una unidad a checkOut en Farma apenas comienza una venta. Primero esperamos que el pago quede correctamente confirmado, para evitar dejar una unidad marcada como despachada en Farma cuando nuestra venta todavía no terminó.